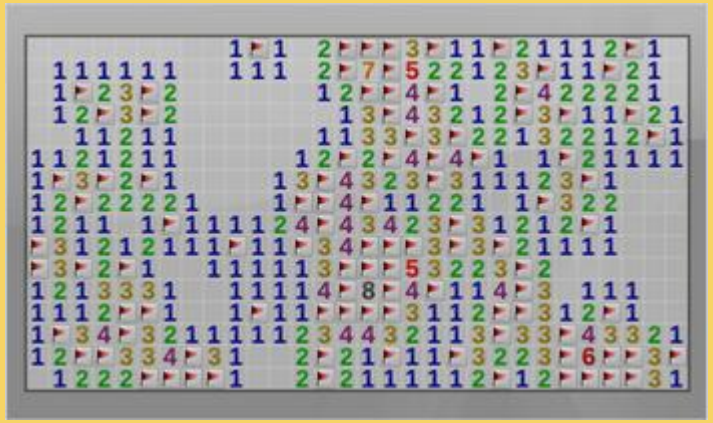


HW 7 coming later this week.

Quiz 9 coming soon, due next Monday.

Class 18: Universal Turing Machine

University of Virginia
CS3120: DMT2
https://weikailin.github.io/cs3120-toc
Wei-Kai Lin

Plan

Computability

Universal Turing Machine

Computable functions

Textbook [TCS] Section 8 and 9

https://introtcs.org/public/lec_07_other_models.html
Equivalent models of computation

Recap: Turing Completeness

Section 8.3.3, Textbook:

A *computational model* M is *Turing complete* if we can map every Turing machine N into a program P for that computes the same function as N.

It is *Turing equivalent* if the other direction holds as well (i.e., we can map every program in M to a Turing machine that computes the same function).

Last Thursday:

Turing Complete

For any computation model X (or programming lang), we say X is *Turing Complete* if and only if X ≡ TM.

Turing complete X includes:

Python, C/C++, Java, …, Haskel, OCaml, lambda-calculus, Conway's Game of Life, …,

18

(Ideal) Python is Turing Equivalent

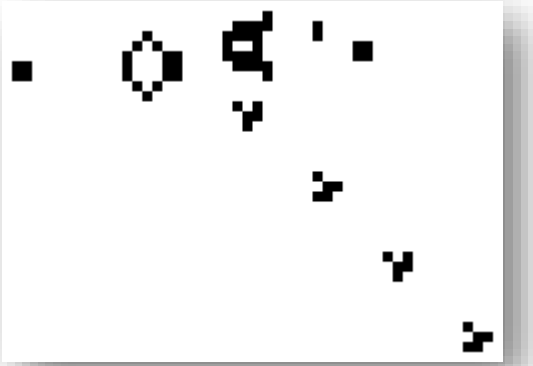
Write Python program (or algorithm in pseudocode), compile it to Turing Machine

Turing Complete Models

Prog lang: Python, C/C++, Haskel, …, lambda-calculus,…

$$\lambda x.(\lambda y.x + y) \quad (8.3)$$

Conway's Game of Life



C++ templates

Meyers, Scott (2005). Effective C++: 55 specific ways to improve your programs and designs

```
template <class T, size_t N>
struct Array;
```

Printf format string

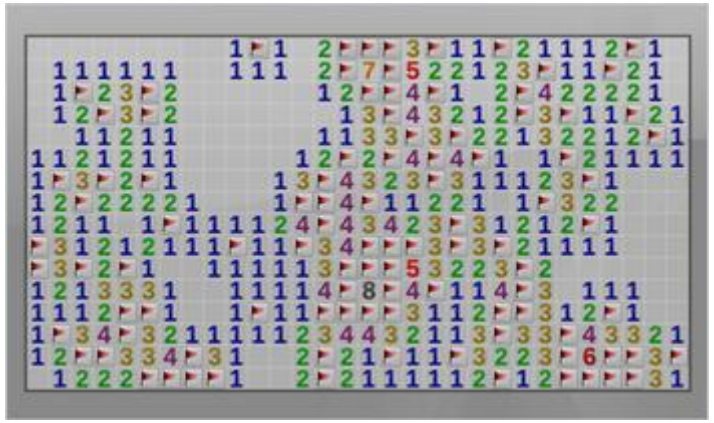
https://dl.acm.org/doi/10.5555/2831143.2831154

```
printf("Your age is %d", age);
```

Microsoft Excel

"Announcing LAMBDA: Turn Excel formulas into custom functions"

Turing-Complete Games



Infinite-grid Minesweeper.
Kaye, Richard (31 May 2007).
https://web.archive.org/web/20160803180423/https://web.mat.bham.ac.uk/R.W.Kaye/minesw/infmsw.pdf

Magic: The Gathering
Magic: The Gathering is Turing Complete. Alex Churchill, Stella Biderman, Austin Herrick. 2019. https://arxiv.org/abs/1904.09828

Is there a model stronger than TM?

A *computational model* M is *Turing complete* if we can map every Turing machine N into a program P for that computes the same function as N.

It is *Turing equivalent* if the other direction holds as well (i.e., we can map every program in M to a Turing machine that computes the same function).

Equivalent question:

Is there any model that is Turing complete but not Turing equivalent?

The Church-Turing Thesis

A function on the natural numbers can be calculated by an effective method if and only if it is computable by a Turing machine.

https://en.wikipedia.org/wiki/Church%E2%80%93Turing_thesis

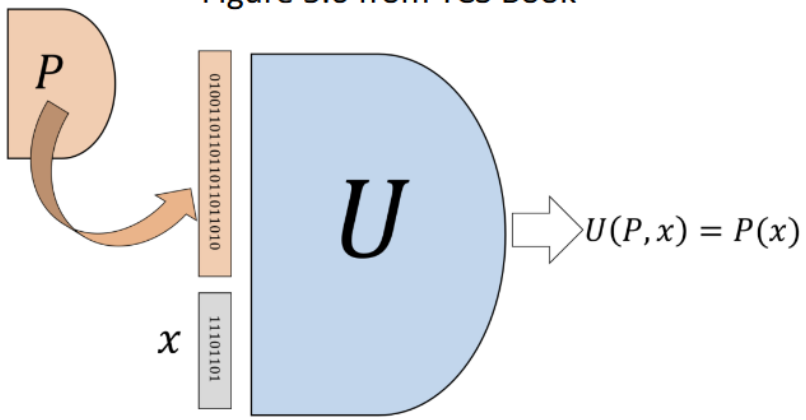
Universal Turing Machine

Recall from Boolean Circuits

From Class 12

Universal Circuit/Program

Figure 5.6 from TCS Book



program U takes a program description P and input x as its input, and "simulates" running P on x:

$$U(P,x) = P(x)$$

Two steps:

1. Represent program as data
2. Build U

Representing Turing Machines

How to represent a Turing machine as a binary string?

- C
- Σ
- δ

$\rightarrow \{0,1\}^*$

A *Turing Machine,* is defined by (Σ, C, δ):

$C \in \mathbb{N}$: a finite number of states

Σ : alphabet $-$ finite set of symbols

$$\Sigma \supseteq \{0, 1, \triangleright, \emptyset\}$$

$\boldsymbol{\delta}$: transition function

$$\boldsymbol{\delta}: [C] \times \Sigma \rightarrow [C] \times \Sigma \times \{\mathbf{L}, \mathbf{R}, \mathbf{S}, \mathbf{H}\}$$

Representing Turing Machines (in Python)

How to represent a Turing machine as a binary string?

- $C = 2$
- $\Sigma = \cancel{\{} 4$
- $\delta =$ 0, 0, 1

```
"""
An example TM.
"""
tm_append = TM(
    states = {'q0', 'q1'},
    transition = {
      1. 'q0': {'0': ('q0', '0', 'R'),
                '1': ('q1', '1', 'R'),
                '▶': ('q0', '▶', 'R'),
                '∅': ('q0', '0', 'H')},
      2. 'q1': {'0': ('q0', '0', 'R'),
                '1': ('q1', '1', 'R'),
                '▶': ('q1', '▶', 'R'),
                '∅': ('q1', '1', 'H')}
    }
)
```

LRHS

unicode/ASCII $\in \{0,1\}^*$

Representing Turing Machines in Binary Strings

Notation: For any $w \in \{0,1\}^*$,

let TM_w be the Turing machine M

such that w represents M.

TM M $\longrightarrow$ $\{0,1\}^*$ w

M' $\xleftarrow{TM_{w'}}$ w'

default $\longleftarrow$ w^*

What if w does not represent any M? $TM_w = ?$

$M(x) = 0$ for all x

Representing Turing Machines in Binary Strings

Notation: For any $w \in \{0,1\}^*$,

let TM_w be the Turing machine M

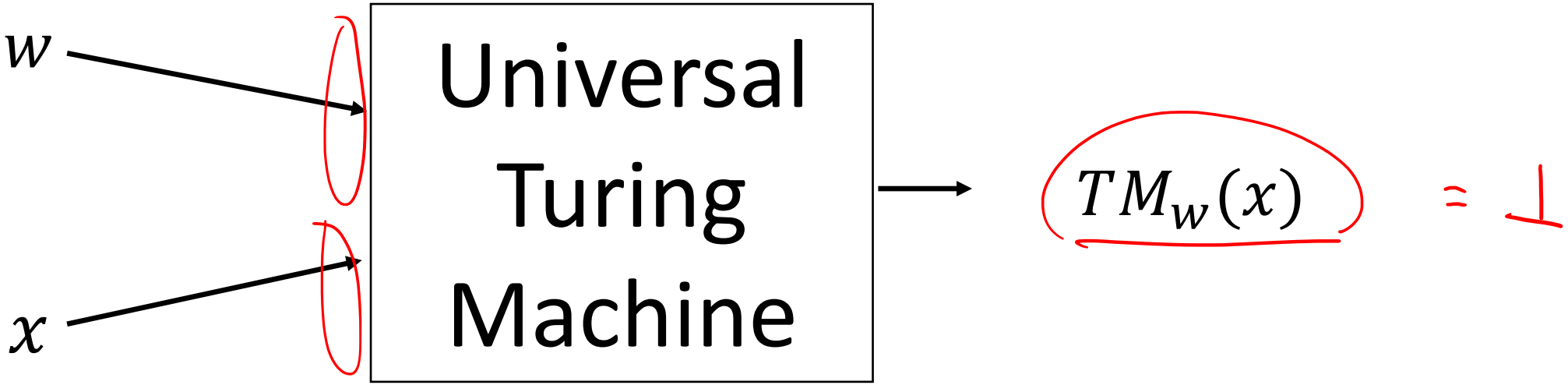
such that w represents M.

What if w does not represent any M?

Convention:

Let TM_w be the TM that output 0 and halt (ignore input).

Is there a "Universal Turing Machine"?



Is there a "Universal (no input) Turing Machine"?

w → Universal Turing Machine → $TM_w(x)$

$x =$ " "

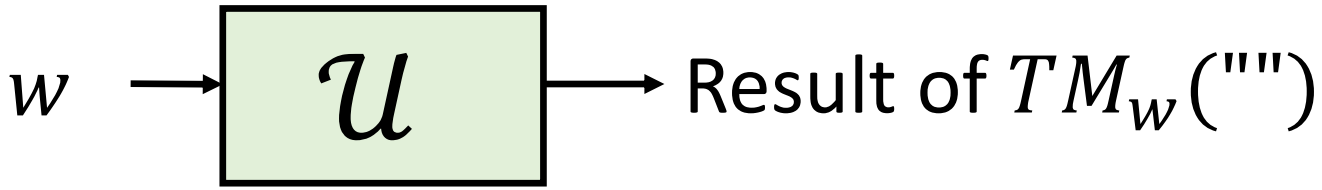
" "

6. *The universal computing machine.*

It is possible to invent a single machine which can be used to compute any computable sequence. If this machine $\mathcal{U}$ is supplied with a tape on the beginning of which is written the S.D of some computing machine $\mathcal{M}$,

then $\mathcal{U}$ will compute the same sequence as $\mathcal{M}$. In this section I explain in outline the behaviour of the machine. The next section is devoted to giving the complete table for $\mathcal{U}$.

What do we need to build $\mathcal{U}$?



$$\mathcal{U}(w) = TM_w(\text{""}).$$

7. *Detailed description of the universal machine.*

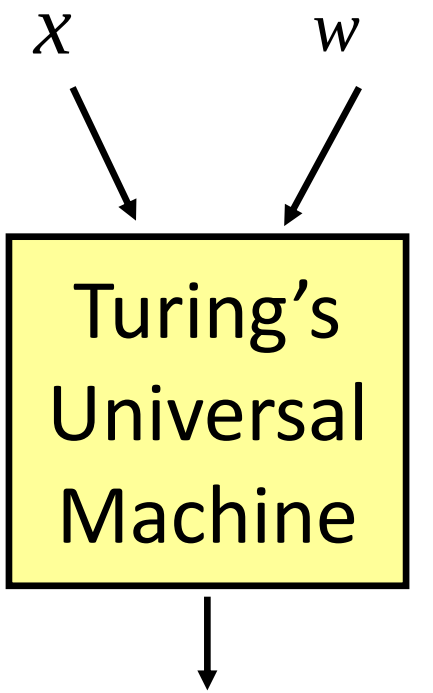
A table is given below of the behaviour of this universal machine. The m-configurations of which the machine is capable are all those occurring in the first and last columns of the table, together with all those which occur when we write out the unabbreviated tables of those which appear in the table in the form of m-functions. *E.g.*, $\mathfrak{e}(\mathfrak{anf})$ appears in the table and is an m-function. Its unabbreviated table is (see p. 239)

$\mathfrak{e}(\mathfrak{anf})$	$\mathfrak{d}$	R	$\mathfrak{e}_1(\mathfrak{anf})$
	not $\mathfrak{d}$	L	$\mathfrak{e}(\mathfrak{anf})$
$\mathfrak{e}_1(\mathfrak{anf})$	Any	R, E, R	$\mathfrak{e}_1(\mathfrak{anf})$
	None		$\mathfrak{anf}$

Consequently $\mathfrak{e}_1(\mathfrak{anf})$ is an m-configuration of $\mathfrak{U}$.

When $\mathfrak{U}$ is ready to start work the tape running through it bears on it the symbol $\mathfrak{d}$ on an F-square and again $\mathfrak{d}$ on the next E-square; after this, on F-squares only, comes the S.D of the machine followed by a double colon "::" (a single symbol, on an F-square). The S.D consists of a number of instructions, separated by semi-colons.

Each instruction consists of five consecutive parts



[Turing 1936]

The table for $\mathfrak{U}$.

$\mathfrak{b}$			$\mathfrak{f}(\mathfrak{b}_1, \mathfrak{b}_1, ::)$
$\mathfrak{b}_1$	$R, R, P:, R, R, PD, R, R, PA$		$\mathfrak{anf}$
$\mathfrak{anf}$			$\mathfrak{g}(\mathfrak{anf}_1, :)$
$\mathfrak{anf}_1$			$\mathfrak{con}(\mathfrak{kom}, y)$
$\mathfrak{kom}$	;	R, Pz, L	$\mathfrak{con}(\mathfrak{kmp}, x)$
	z	L, L	$\mathfrak{kom}$
	not z nor ;	L	$\mathfrak{kom}$
$\mathfrak{kmp}$			$\mathfrak{cpe}\big(\mathfrak{c}(\mathfrak{kom}, x, y), \mathfrak{sim}, x, y\big)$

$\mathfrak{b}$. The machine prints $:DA$ on the F-squares after $:: \rightarrow \mathfrak{anf}$.

$\mathfrak{anf}$. The machine marks the configuration in the last complete configuration with y. $\rightarrow \mathfrak{kom}$.

$\mathfrak{kom}$. The machine finds the last semi-colon not marked with z. It marks this semi-colon with z and the configuration following it with x.

$\mathfrak{kmp}$. The machine compares the sequences marked x and y. It erases all letters x and y. $\rightarrow \mathfrak{sim}$ if they are alike. Otherwise $\rightarrow \mathfrak{kom}$.

Tape

[L+,0R.,1R.!1L+,1L+,0L.:,0L.,1L.:]1011

Head

Current state

0

Universal Turing machine successfully loaded

Steps

0

Turing machine program

```
; ---------------------------------------------------------------
; A Universal Turing Machine
; ---------------------------------------------------------------
; For use with Turing machine simulator http://morphett.info/turing/turing.html.
; ---------------------------------------------------------------
; David Bevan
; The Open University, England
; April 2016
; http://mathematics.open.ac.uk/people/david.bevan
; ---------------------------------------------------------------
; This UTM simulates 3-symbol Turing machines whose symbol set is {blank, 0, 1}.
; ---------------------------------------------------------------
; A specification of the input format can be found in the file utm.pdf
; in the Dropbox folder linked from http://tinyurl.com/M269resources.
; ---------------------------------------------------------------
; Example inputs:
;   Binary parity bit
;     [0L++, R., R+!1L+, R., R-:,,:]01011
;   Binary increment
;     [ L+,0R.,1R.!1L+,1L+,0L.:,0L.,1L.:]1011
;   Unary subtraction
;     [ L+,,1R.! L.,, R+: R.,, R+:,,1L--:]11111 11
;   Binary palindrome detector
;     [ R+++++++  R+  R++! L++ 0R  1R : L++ 0R  1R : R++++  L++  L+++++: R+++  L++++  L+: R++ 0L
```

Controls

Run

Run at full speed

Pause

Step

Undo

Reset

Initial input: [L+,0R.,1R.!1L+,1L+,0L.:,

Advanced options

Load an example program

Save to the cloud

Universal TM in Python:

(Class 17: can compile Python to RAM to TM)

```python
"""
The execution of a TM performs
1. Initialize the tape with '▶' + str_input + '∅'
2. Execute the transition function, util the halting 'H' or the max_step
3. Output the tape from the first symbol to the next none 0/1 symbol
"""
def execute(self, str_input, max_step = 5000):
    # Initialize the tape with '▶' + str_input + '∅'
    tape = list('▶' + str_input + '∅') # Convert to list for mutability
    # Execute the transition function, util the halting 'H' or the max_step
    cur_state = self.initial
    head = 0
    step_i = 0
    next_dir = ''
    for step_i in range(max_step):
        assert self.transition[cur_state].keys() == self.alphabets
        assert head < len(tape)
        assert tape[head] in self.alphabets
        (next_state, write_symb, next_dir) = self.transition[cur_state][tape[head]]
        cur_state = next_state
        tape[head] = write_symb
        if next_dir == 'L':
            head -= 1
            if head < 0:
                head = 0
        elif next_dir == 'R':
            head += 1
            if head >= len(tape) - 1:
                tape.append('∅')      ...
```

Computability

ON COMPUTABLE NUMBERS, WITH AN APPLICATION TO THE ENTSCHEIDUNGSPROBLEM

By A. M. Turing.

[Received 28 May, 1936.—Read 12 November, 1936.]

The "computable" numbers may be described briefly as the real numbers whose expressions as a decimal are calculable by finite means. Although the subject of this paper is ostensibly the computable *numbers*, it is almost equally easy to define and investigate computable functions of an integral variable or a real or computable variable, computable predicates, and so forth. The fundamental problems involved are however, the same in each case, and I have for explicit treatment as involving the le shortly to give an account of the relati functions, and so forth to one another. of the theory of functions of a real vari putable numbers. According to my definition, a number is computable if its decimal can be written down by a machine.

Roughly: numbers that can be output *arbitrarily precisely* by a Turing Machine (No Input model).

Computable number:

A real number a is computable iff there exists a Turing Machine M such that for all $i \in \mathbb{N}$, $M(i)$ outputs the ith digit after the decimal point.

a

Examples:

1/2 = 0.50000… $\qquad M(1) = 5, M(2) = 0, M(3) = 0, \ldots$

1 2 3 4 5 ...

1/3 = 0.33333… $\qquad M(1) = 3, M(2) = 3, \ldots$

π = 3.141592… = 4 $(1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \frac{1}{9} - \frac{1}{11} + \cdots)$

$M(1) = 1,$

$M(2) = 4,$ compute by first 1000 terms in the series

$M(3) = 1,$ compute by first 10000 terms in the series

$M(4) = 5,$ compute by first 100000 terms in the series

$\sqrt{2}$ = 1.414213…

Are there any *uncomputable* numbers?

How many *computable* numbers are there?

The "computable" numbers may be described briefly as the real numbers whose expressions as a decimal are calculable by finite means.

How many Turing Machines are there?

A *Turing Machine,* is defined by (Σ, C, δ):

$C \in \mathbb{N}$: a finite number of states

Σ : alphabet – finite set of symbols

$$\Sigma \supseteq \{0, 1, \triangleright, \emptyset\}$$

$\boldsymbol{\delta}$: transition function

$$\boldsymbol{\delta}: [C] \times \Sigma \to [C] \times \Sigma \times \{\mathbf{L}, \mathbf{R}, \mathbf{S}, \mathbf{H}\}$$

infinite $\geq |\mathbb{N}|$

Countable

$|TM| \to \{0,1\}^*$

Countable

$|TM| \leq |\{0,1\}^*|$

Countable

Some Numbers are Uncomputable!

$$|\,TMs\,| \;=\; |\,\text{finite binary strings}\,| \;=\; |\,\mathbb{N}\,|$$

$$|\,\mathbb{R}\,| = |\,pow(\mathbb{N})| > |\,\mathbb{N}\,|$$

$\exists\ a \in \mathbb{R}$ but no TM s.t comp a.

Is there an *interesting* number that cannot be computed?

bits

A real number is a Boolean function:
For each $a \in [0,1)$, represent $a = 0.a_0a_1a_2a_3 \ldots$.
Let $f_a : \{0,1\}^* \to \{0,1\}$ be the following:
$f_a(x) = a_i$, where $x \in \{0,1\}^*$ represents $i \in \mathbb{N}$.

$f(i) =$ ith bit of a

Is there an *interesting* Boolean function that cannot be computed?

Are there *functions* that cannot be computed by any Turing Machine?

Computable Functions

Definition:

A Boolean function $F: \{0,1\}^* \to \{0,1\}$ is ***computable*** if and only if there exists a Turing machine M such that for all $x \in \{0,1\}^*$, $M(x) = F(x)$.

When the function F is Boolean (output 1 bit), we sometimes say F is **decidable,** which stresses both computable and Boolean output.
When F **outputs many bits**, we also say F is computable iff such TM M exists.

Is there an *interesting function* that cannot be computed? (Uncomputability)

Logistics

- Free food/drink at Greenberry's.

HW 7 coming later this week.

Quiz 9 coming soon, due next Monday.

Plan

Computability

Halting Problem

Rice's Theorem

Textbook [TCS] Section 9

https://introtcs.org/public/lec_08_uncomputability.html
Universality and uncomputability